GEN AI LEVEL 2 — AGENTIC AI MASTER GUIDE

44-Day Mysore Program: LangGraph, LiteLLM Gateway, Model Context Protocol (MCP) & HITL

---

1. Program Overview & Architectural Scope

GenAI Level 2 is an intensive 44-day masterclass on autonomous cognitive architectures and multi-agent systems. Students progress beyond linear chains into cyclical stateful graphs with LangGraph, implement LLM gateways using LiteLLM, connect dynamic tools using the Model Context Protocol (MCP), establish Human-in-the-Loop (HITL) safety gates, and orchestrate automated CI/CD deployment pipelines.

2. Course Modules & Timeline (44 Days Breakdown)

S No.	Course Module Name	Duration	Core Pedagogical Focus
1	Agentic AI Foundation & Design Patterns	2 Days	ReAct, Plan-and-Solve, Reflection, Critic loops, supervisor topologies
2	Programming Framework for Agentic System	5 Days	CrewAI, AutoGen, LangGraph primitives, state machines, nodes & ed
3	Agent Persona Modeling, Execution & Inferencing	2 Days	Cognitive boundaries, prompt steering, deterministic vs creative tunin
4	Agent Knowledge Base & Human in Loop	5 Days	Hybrid search, memory stores, approval gates, state rewind & human
5	DevOps for Agentic AI Applications	3 Days	Containerizing agents, async queue workers, environment isolation.
6	Agent Tool Discovery & Communication	8 Days	Function calling schemas, Model Context Protocol (MCP), inter-agent
7	Agent Observability	3 Days	OpenTelemetry, token tracking, latency profiling, cost attribution, eval
8	Deploying Agents in Automated CI/CD Pipeline	3 Days	GitHub Actions, prompt regression tests, automated evaluation, Dock
9	LiteLLM, LLM Gateway & Agent Lifecycle	3 Days	LiteLLM proxy routing, model fallbacks, rate-limiting, budget threshold
10	LangGraph State Mgmt & Control Flow	7 Days	StateGraph, conditional routing, supervisor-worker patterns, checkpo
11	Agent Persona Modeling & Inferencing	3 Days	Specialized multi-agent squad synthesis, edge case validation & final

3. Production Capstone Project: AgenticDevOps Mesh

Repository: https://github.com/ShreyasVavley/genai-l2-agentmesh

AgenticDevOps Mesh is an autonomous multi-agent platform for cloud software engineering and continuous delivery. It coordinates specialized agent personas (LeadArchitect, SecurityAuditor, DevOpsEngineer, CriticReviewer) through a LangGraph state graph. Dangerous shell executions trigger an interactive Human-in-the-Loop interrupt gate.

Key Technological Implementations:

• **LangGraph StateGraph:** TypedDict state persistence with conditional edge routing and quality reflection.

• **LiteLLM Gateway Router:** Centralized routing across models (Gemini, GPT-4, Claude) with fallback logic and budget metrics.

• **Model Context Protocol (MCP):** Dynamic discovery catalog exposing Git, Dockerfile, VulnScanner, and Shell tools.

• **Human-in-the-Loop (HITL) Gate:** Interrupt state halting execution before destructive operations until human approval is delivered.

• **OpenTelemetry Observability:** Distributed trace spans, latency measurements, and token expenditure accounting.

Sample LangGraph StateGraph Architecture Code:

```
# Production LangGraph Multi-Agent StateGraph
builder = StateGraph(AgentMeshState)
builder.add_node("architect", architect_node)
builder.add_node("security", security_node)
builder.add_node("devops", devops_node)
builder.add_node("critic", critic_node)
builder.add_node("hitl_gate", hitl_gate_node)
builder.add_node("executor", execution_node)

builder.add_edge("architect", "security")
builder.add_edge("security", "devops")
builder.add_edge("devops", "critic")
builder.add_edge("critic", "hitl_gate")
builder.add_conditional_edges("hitl_gate", should_execute, {
    "executor": "executor",
    END: END
})
```

4. Student Hands-on Lab Drills & Weekly Milestones

Weeks 1-2 (Days 1-9): Agent Foundations & Persona Engineering

Lab 1: Design system prompts and cognitive bounds for a multi-agent team.
Lab 2: Implement Plan-and-Solve and Reflection loops in pure Python.

Weeks 3-4 (Days 10-17): State Machines, LangGraph & HITL Gates

Lab 3: Build a LangGraph StateGraph with custom TypedDict schemas.
Lab 4: Implement state checkpoints and human interruption gates before critical actions.

Weeks 5-6 (Days 18-28): Dynamic MCP Tools & Agent Observability

Lab 5: Register custom tools conforming to Model Context Protocol (MCP) JSON specifications.
Lab 6: Integrate OpenTelemetry spans to measure per-agent latency and token consumption.

Weeks 7-9 (Days 29-44): LiteLLM Gateway, CI/CD & Final Capstone Defense

Lab 7: Configure LiteLLM proxy with fallback models and budget thresholds.
Lab 8: Deploy the multi-agent mesh to Docker/Kubernetes and automate regression testing via GitHub Actions.

5. Advanced Agentic Technical Interview Question Bank

Q1: How does LangGraph solve the limitations of standard linear DAG chains for agentic workflows?

Answer: Real-world agentic problems require cyclical loops (reflection, retries, multi-turn tool calling) and persistent state management. Standard DAG chains cannot loop back to previous nodes, whereas LangGraph natively supports cyclical graphs, conditional edges, and state checkpointing.

Q2: What is the significance of the Model Context Protocol (MCP)?

Answer: MCP standardizes how AI agents discover and invoke external tools and resources across different platforms, replacing proprietary ad-hoc tool formatting with a universal, secure protocol.

Q3: How do you implement Human-in-the-Loop (HITL) without blocking server worker threads?

Answer: State is persisted into a database or checkpoint store (e.g. SQLite, PostgreSQL, Redis) with a status of 'waiting_for_approval'. The server worker releases memory and exits. When the human approves via REST API, the state is retrieved and execution resumes.